

CENG418 — Deep Learning

Course Project Report

Fashion-MNIST Image Classification with a Convolutional Neural Network

A Three-Way Ablation Study:
Isolating the Effects of Architecture and Data Augmentation

Author: Cemal Özcan

Student ID: 220401094

Instructor: Çağdaş Eşiyok

Date: May 2026

1. Introduction

This project trains a Convolutional Neural Network (CNN) on the Fashion-MNIST dataset and reports the results as a three-way ablation study. Instead of evaluating a single model, three models are trained to isolate the individual contributions of (i) the convolutional architecture and (ii) data augmentation:

- Baseline: dense (fully-connected) network — no convolutions, no augmentation.
- Plain CNN: CNN architecture, no augmentation.
- Augmented CNN: identical CNN architecture, with data augmentation at the input.

The transitions Baseline → Plain CNN and Plain CNN → Augmented CNN respectively quantify the architecture and augmentation contributions. The most interesting result of the study is an unexpected one: under the chosen settings, augmentation did not improve performance — and the report explains why this is consistent with the dataset's properties rather than a flaw in the setup.

2. Problem and Dataset

The task is multi-class image classification: given a 28×28 grayscale image of a clothing item, predict its category among 10 possible classes. Fashion-MNIST contains 70,000 such images and is intended as a more challenging drop-in replacement for the original MNIST (handwritten digits).

Split	Images	Purpose
Training set	60,000	Learning
– Training portion (90%)	54,000	Direct optimization
– Validation portion (10%)	6,000	Monitoring during training
Test set	10,000	Final generalization evaluation
TOTAL	70,000	—

The 10 classes are:

Label	Class	Label	Class
0	T-shirt/top	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag

4	Coat	9	Ankle boot
---	------	---	------------



Figure 1. Random samples from the training set, with the true class label below each image.

3. Methodology

3.1 Preprocessing

Normalization: pixel intensities are rescaled from $[0, 255]$ to $[0, 1]$, which improves numerical stability and convergence speed.

Reshape: images are reshaped from $(28, 28)$ to $(28, 28, 1)$ so that Conv2D layers receive an explicit channel dimension.

3.2 Baseline Model (Dense NN)

A simple fully-connected network is used as a reference. It does not exploit any spatial structure — pixels are flattened into a 784-dim vector and processed by two dense layers.

#	Layer	Output
1	Flatten	784
2	Dense (128, ReLU)	128
3	Dropout (0.3)	128
4	Dense (10, Softmax)	10

The baseline is trained for 5 epochs only. The goal is to obtain a reference accuracy, not to tune a competitive model.

3.3 Plain CNN (no augmentation)

The Plain CNN uses convolution and pooling layers to exploit spatial structure but does not apply data augmentation. Training the Plain CNN as a separate model — with an identical architecture to the Augmented CNN below — is what enables the ablation: comparing it against the Baseline isolates the architecture's contribution, and comparing it against the Augmented CNN isolates the augmentation's contribution.

3.4 Data Augmentation

Three random transformations are applied at training time only (disabled at inference) to encourage robustness:

Transform	Parameter	Description
RandomRotation	0.05 ($\approx \pm 18^\circ$)	Random rotation
RandomZoom	0.08 ($\pm 8\%$)	Zoom in / out
RandomTranslation	0.03 / 0.03	Horizontal / vertical shift

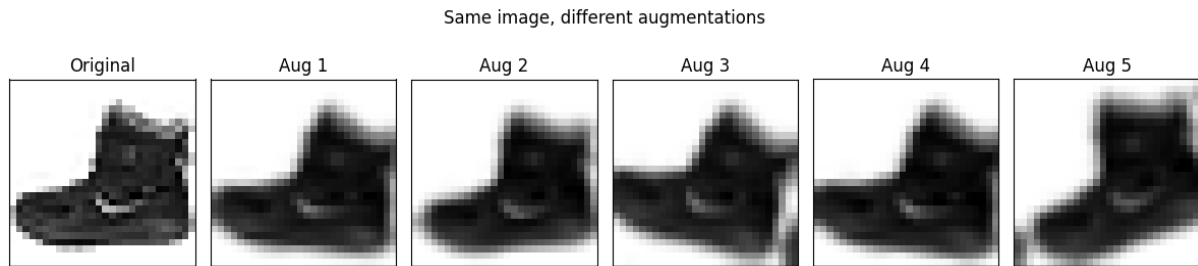


Figure 2. The original image (left) and five augmented versions produced by the augmentation layer.

3.5 CNN Architecture

Both the Plain CNN and the Augmented CNN use the architecture below. The only difference is whether the augmentation block at the input is active.

#	Layer	Output Shape	Role
1	Data Augmentation (Aug CNN only)	(28, 28, 1)	Random transforms
2	Conv2D (32 filters, 3×3)	(26, 26, 32)	Low-level features
3	MaxPooling2D (2×2)	(13, 13, 32)	Downsample
4	Conv2D (64 filters, 3×3)	(11, 11, 64)	Mid-level patterns
5	MaxPooling2D (2×2)	(5, 5, 64)	Downsample
6	Conv2D (64 filters, 3×3)	(3, 3, 64)	High-level features
7	Flatten	576	2D → 1D
8	Dense (64, ReLU)	64	Fully connected
9	Dropout (0.5)	64	Regularization
10	Dense (10, Softmax)	10	Class probabilities

4. Mathematical Background

This section briefly summarizes the operations used in the model. The aim is to make the design choices traceable rather than treating the architecture as a black box.

4.1 2D Convolution

For an input I and a kernel K , 2D discrete convolution is defined as:

$$(I * K)(i, j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m, n)$$

Each Conv2D layer applies many such kernels in parallel; each kernel is responsible for detecting a specific local pattern (an edge, a texture, etc.).

4.2 ReLU Activation

After every convolution, a ReLU non-linearity is applied:

$$f(x) = \max(0, x)$$

ReLU is computationally cheap and largely sidesteps the vanishing-gradient issue that affects sigmoid / tanh in deep networks.

4.3 Max Pooling

MaxPooling2D with a 2×2 window keeps the maximum within each window:

$$\text{MaxPool}(X)_{i,j} = \max\{ X(2i+a, 2j+b) : a, b \in \{0, 1\} \}$$

Pooling halves spatial resolution, reducing compute and providing some translation invariance.

4.4 Softmax Output

Logits z are converted to a probability distribution over the 10 classes:

$$\sigma(z)_i = e^{z_i} / \sum_j e^{z_j}$$

4.5 Categorical Cross-Entropy Loss

The loss compares predicted distribution $\hat{y}$ against the one-hot true label y :

$$L(y, \hat{y}) = - \sum_i y_i \cdot \log(\hat{y}_i)$$

When labels are stored as integers, the equivalent sparse form (`sparse_categorical_crossentropy`) is used in the implementation.

4.6 Adam Optimizer

Adam maintains exponential moving averages of the first (m) and second (v) moments of past gradients. With $\beta_1=0.9$, $\beta_2=0.999$ and $\epsilon=10^{-8}$, parameters are updated as:

$$\begin{aligned}m_t &= \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t \\v_t &= \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2 \\\theta_t &= \theta_{t-1} - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)\end{aligned}$$

where g_t is the current gradient and η is the learning rate. $\hat{m}$ and $\hat{v}$ are bias-corrected versions of the moment estimates. The key feature of Adam is that it computes a per-parameter effective learning rate.

5. Training Hyperparameters

Hyperparameter	Baseline	Plain CNN	Aug CNN	Notes
Optimizer	Adam	Adam	Adam	Default settings
Loss	Sparse CCE	Sparse CCE	Sparse CCE	Multi-class
Epochs	5	15	30	Aug CNN sees a different sample each epoch
Batch size	64	64	64	—
Validation split	0.1	0.1	0.1	10% of train held out

6. Results — Three-Way Comparison

All three models are evaluated on the same held-out test set (10,000 images). The headline result is summarized below.

Model	Test Accuracy	Parameters	Training Time (GPU)
Baseline (Dense NN)	87.02%	101,770	≈ 15 s
Plain CNN (no aug)	90.58% □	93,322	≈ 90 s
Augmented CNN	89.19%	93,322	≈ 5 min
Best result (Plain CNN)	+3.56 pts	Fewer params!	—

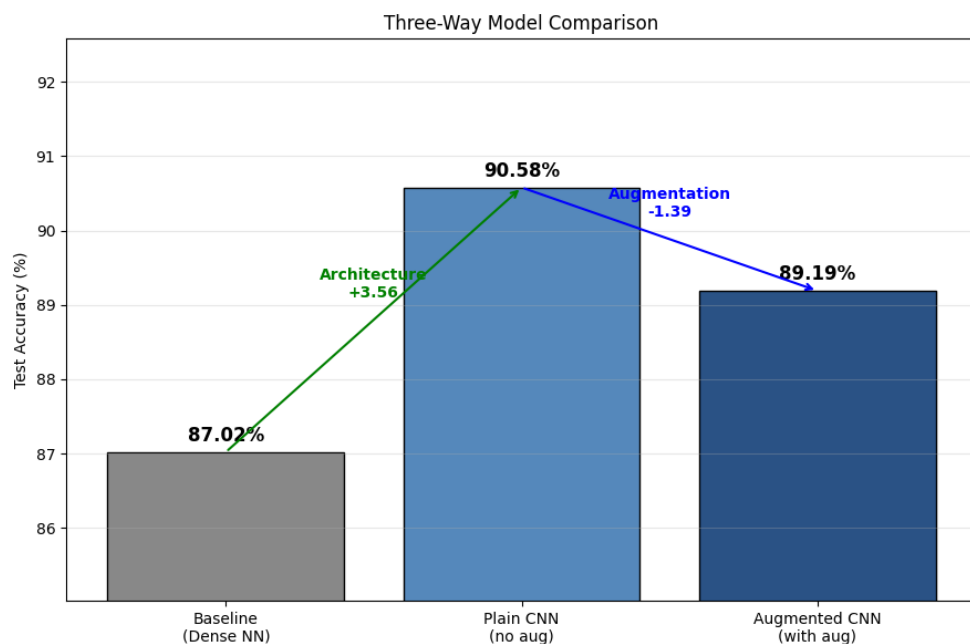


Figure 3. Three-way test accuracy comparison. The green arrow shows the architecture's contribution (+3.56 points); the blue arrow shows the augmentation contribution (−1.39 points).

The CNN models use fewer parameters than the Baseline ($\approx 93\text{K}$ vs. $\approx 102\text{K}$) yet outperform it. This is a direct illustration of the parameter efficiency of convolutional architectures: weight sharing across spatial positions captures more useful structure per weight.

6.1 Contribution Decomposition

The two transitions in the ablation map directly onto two design decisions:

Transition	Variable changed	Effect
Baseline $\rightarrow$ Plain CNN	Architecture (Dense $\rightarrow$ CNN)	+3.56 pts
Plain CNN $\rightarrow$ Augmented CNN	Augmentation (off $\rightarrow$ on)	-1.39 pts
Total (Baseline $\rightarrow$ Best)	—	+3.56 pts

Architecture (+3.56 pts). Convolution and pooling allow the network to exploit spatial locality directly, so the same representational power is achieved with fewer parameters than a fully connected network would need.

Augmentation (-1.39 pts). Contrary to the common assumption that augmentation always helps, in this experiment it slightly hurt test accuracy. Likely reasons:

- The dataset is already large enough. 60,000 balanced training samples provide enough variety on their own. The Plain CNN's training and validation curves stay close (train $\approx 93\%$, val $\approx 91\%$), indicating no overfitting in the first place.
- Augmentation can act as noise rather than regularization. If the model is not overfitting, additional input perturbations distort signal that the model would otherwise learn cleanly.
- Selective applicability of augmentation. Augmentation is most beneficial when (a) the dataset is small or (b) the model is overfitting. Neither holds strongly here.

This is consistent with the broader literature: data augmentation is a useful regularizer in many settings, but its effect must be evaluated empirically rather than assumed.

6.2 Training Curves

Examined separately, both CNNs show small train/val gaps. The Plain CNN reaches its plateau quickly without overfitting, which leaves little room for augmentation to improve performance further.

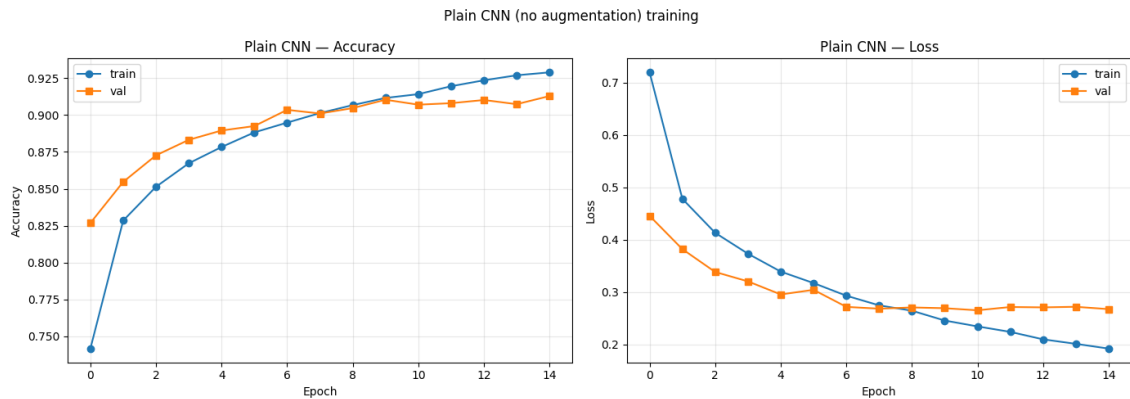


Figure 4a. Plain CNN — accuracy and loss curves over 15 epochs. Train and validation accuracy stay close, indicating no overfitting.

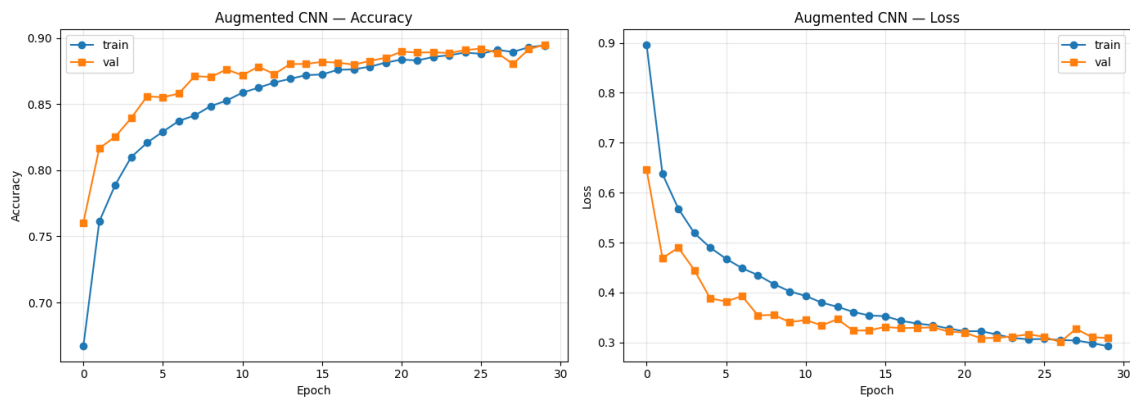


Figure 4b. Augmented CNN — accuracy and loss curves over 30 epochs. Convergence is slower because each epoch presents a different augmented sample.

7. Error Analysis

Test accuracy alone is insufficient — the qualitative pattern of the model's mistakes is just as informative. The analysis below uses the Augmented CNN's predictions; the same patterns are visible in Plain CNN, since the errors are dataset-driven rather than model-specific. Out of 10,000 test samples, 1,081 (10.81%) are misclassified.

7.1 Sample Predictions

The figure below shows 15 random predictions from the test set: blue labels are correct, red labels are wrong. The percentage shows the model's confidence in its top prediction.



Figure 5. Sample predictions on the test set. Blue: correct; Red: incorrect.

7.2 Confusion Matrix

Diagonal entries are correct predictions; off-diagonal hotspots reveal systematic confusions.

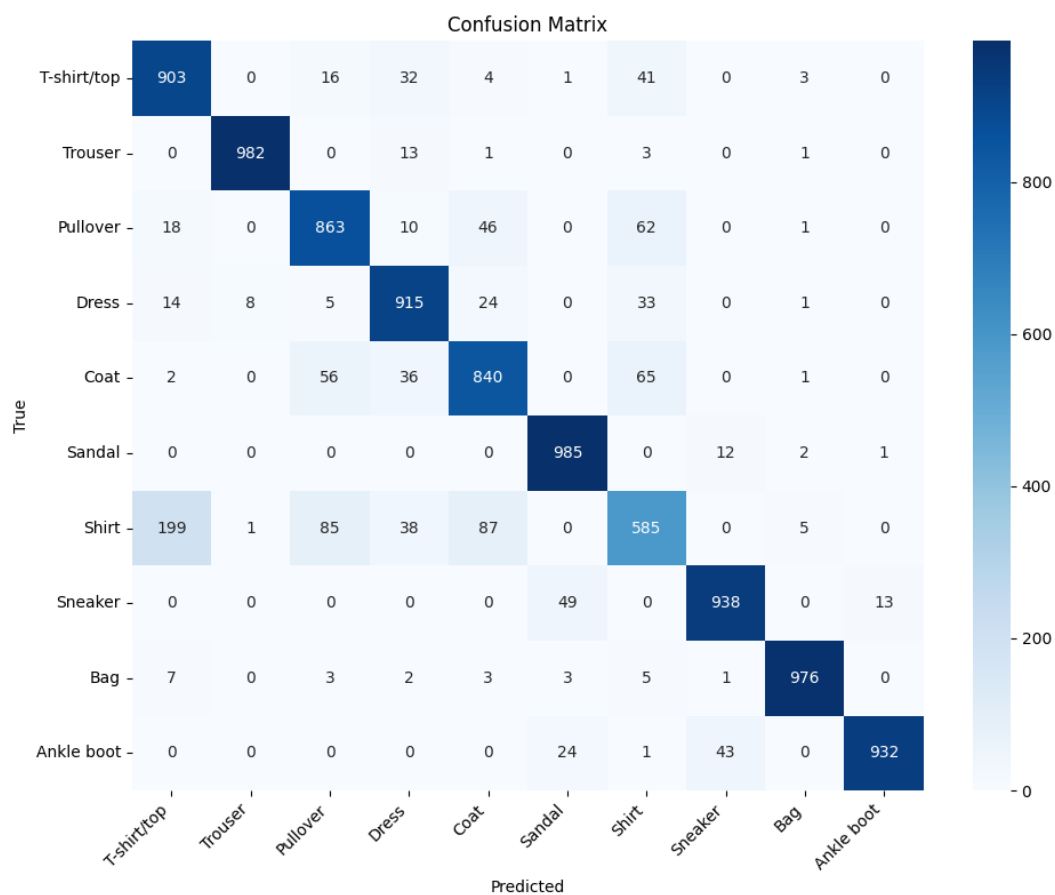


Figure 6. Confusion matrix of the Augmented CNN. The Shirt row stands out as the main source of confusion.

7.3 Most Confused Class Pairs

The five largest off-diagonal confusion counts are:

True Class	→	Predicted As	Count	Comment
Shirt	→	T-shirt/top	199	Both short-sleeved tops
Shirt	→	Coat	87	Long vs. short sleeve confusion
Shirt	→	Pullover	85	Similar upper-body silhouette
Coat	→	Shirt	65	Reverse direction of the same overlap
Pullover	→	Shirt	62	Reverse direction of the same overlap

All five hotspots involve the Shirt class. Per-class results confirm this: Shirt has the lowest recall (≈ 0.58) of all classes, while visually distinct classes (Trouser, Bag, Sneaker, Ankle boot) achieve recall ≥ 0.93 . The mistakes are not random — they are concentrated where two classes look genuinely similar at 28×28 grayscale, where even a human annotator would struggle. The model has learned meaningful visual features.

7.4 Most Confidently Wrong Predictions

Below are 12 misclassifications where the model assigned the highest probability to the wrong class — these are the most informative errors. Many of them are borderline cases that arguably mislead even a human labeler.

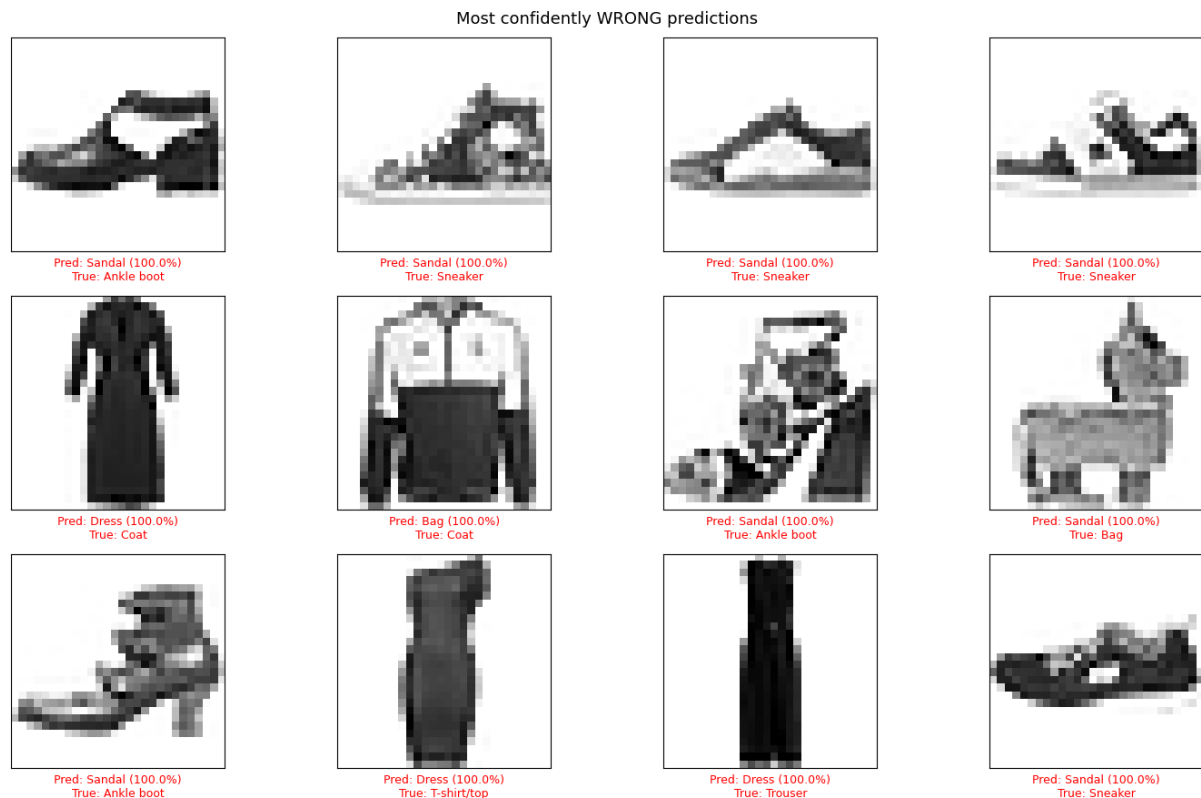


Figure 7. The 12 most confidently incorrect predictions. Below each image: the predicted class with confidence, and the true label.

7.5 Possible Improvements

- Higher resolution images. 28×28 grayscale loses fine-grained details that distinguish similar items (e.g., a button row on a shirt vs. a smooth pullover).
- RGB color information. Texture and color cues would help separate visually similar silhouettes.
- Transfer learning. A pre-trained backbone (e.g., ResNet) fine-tuned on Fashion-MNIST would benefit from features learned on much larger datasets.
- Class-aware training. Class weighting or focal loss could push the model to invest more capacity in the hard Shirt class.
- Early stopping / learning-rate scheduling. Useful in general, although the curves here suggest there is little additional gain to be made on this benchmark with this architecture.

8. Conclusion

This project applied a three-way ablation study to the Fashion-MNIST image classification problem and reports the following main findings.

- (1) The best model is the Plain CNN with a test accuracy of 90.58%, an improvement of +3.56 points over the Dense baseline (87.02%). This is concrete evidence of the value of using a convolutional architecture for image data.
- (2) The CNN models achieve this gain with fewer trainable parameters than the baseline ($\approx 93\text{K}$ vs. $\approx 102\text{K}$), illustrating the parameter efficiency that comes from weight sharing.
- (3) Unexpected finding. Data augmentation did not help — it reduced test accuracy by 1.39 points. The Plain CNN's training and validation curves stay close (no overfitting), so augmentation could not function in its usual regularizer role and acted as noise instead. This is a concrete counter-example to the common assumption that augmentation is universally beneficial.
- (4) The model's errors are not random; they cluster around the visually similar classes (Shirt $\leftrightarrow$ T-shirt, Coat $\leftrightarrow$ Pullover). All five top off-diagonal entries in the confusion matrix involve Shirt, and Shirt is the only class with recall < 0.7 . This indicates that the model has learned meaningful visual features.
- (5) The broader takeaway is that design decisions in deep learning — including those usually treated as 'best practice' — should be validated empirically against the specific dataset and model rather than assumed.

9. References

- Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv preprint arXiv:1708.07747.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436–444. doi:10.1038/nature14539.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press, Cambridge, MA.
- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.
- Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on Image Data Augmentation for Deep Learning. *Journal of Big Data*, 6, Article 60. doi:10.1186/s40537-019-0197-0.

- Chollet, F. (2015). Keras. <https://keras.io>.